

# Навчальний посібник: Парадигми типізації в IT

Підготовка до іспиту

16 травня 2026 р.

## Загальні відомості

---

- **Предмет:** Основи програмування та комп'ютерних наук
- **Тема:** Типізація мов програмування

## Анотація

---

Цей посібник охоплює фундаментальні принципи того, як мови програмування обробляють типи даних. Розуміння цих концепцій є критичним для розробників, QA-інженерів та DevOps-фахівців, оскільки це впливає на виявлення помилок, стабільність системи та швидкість розробки.

## Ключові концепції

---

### 1. Час перевірки типів

- **Статична типізація (Java, C#):** Перевірка типів відбувається на етапі компіляції. Якщо ви намагаєтесь присвоїти рядок змінній типу `int`, програма просто не збереться.
- **Динамічна типізація (Python, JS):** Перевірка відбувається під час виконання програми. Це дає гнучкість, але підвищує ризик помилок "в руках" користувача.

### 2. Строгість правил

- **Сильна (Строга) типізація (Python, Java):** Мова забороняє автоматично змінювати типи. Операція `5 + "10"` призведе до помилки.
- **Слабка типізація (JS, C):** Мова автоматично приводить типи. JavaScript може перетворити `5 + "10"` на `"510"`.

## Порівняльна таблиця популярних мов

---

## Словник термінів

---

**Runtime:** Період, коли програма безпосередньо працює.

**Compile-time:** Етап перетворення коду в програму.

| Мова       | Перевірка (Стат/Дин) | Строгість (Сил/Слаб)             |
|------------|----------------------|----------------------------------|
| Java       | Статична             | Сильна                           |
| Python     | Динамічна            | Сильна                           |
| C++        | Статична             | Слабка (через роботу з пам'яттю) |
| JavaScript | Динамічна            | Слабка                           |

**Implicit Type Conversion:** Автоматичне (неявне) перетворення типів.

**TypeError:** Помилка, що виникає при спробі виконати недопустиму операцію з типом.

**Lint:** Інструмент для пошуку помилок у код без його запуску.

### Питання для самоперевірки

1. Чому Python вважається сильно типізованим, хоча ми не пишемо типи змінних?
2. У чому головна небезпека слабкої типізації для фінансових систем?
3. Як статична типізація допомагає у великих командах розробників?